

Práctica 3: Raspberry Pi

Estiven Castrillon Álzate, Ana Sofia Mora Carmona, Ana María Hurtado Roldán
Estudiante de Física, Universidad de Antioquia
Medellín, Colombia

Resumen—Este documento presenta el desarrollo y los resultados de la Práctica 3 utilizando la Raspberry Pi 4 Model B para la implementación de un modelo de clasificación de imágenes basado en aprendizaje automático. Se detalla la configuración del sistema operativo Raspberry Pi OS de 64 bits, la preparación del entorno virtual en Python y la integración de herramientas como OpenCV, TensorFlow y Google Teachable Machine. El modelo fue entrenado para clasificar tres categorías (Cargador, Memoria USB y Fondo), evaluando métricas críticas de rendimiento como nivel de confianza, latencia de inferencia y consumo de recursos de hardware (CPU y RAM).

Index Terms—Raspberry Pi 4B, TensorFlow, OpenCV, Aprendizaje Automático, Sistema Embebido.

I. INTRODUCCIÓN

La Raspberry Pi 4 Model B es una computadora de placa única desarrollada por la Fundación Raspberry Pi, una organización benéfica con sede en el Reino Unido cuyo propósito fue hacer la computación y la creación digital accesibles para todos [1]. A pesar de su reducido tamaño, comparable al de una tarjeta de crédito, integra todos los componentes esenciales de una computadora convencional: un procesador de cuatro núcleos de 64 bits, soporte para pantallas duales en resoluciones de hasta 4K a través de puertos micro-HDMI, decodificación de video por hardware, hasta 8 GB de RAM, conectividad inalámbrica de doble banda, Bluetooth 5.0, Ethernet Gigabit y USB 3.0 [2]. Su funcionamiento se basa en un sistema operativo almacenado en una tarjeta microSD, desde la cual arranca al recibir alimentación eléctrica, operando de manera análoga a cualquier computador de escritorio comparable a sistemas x86 de nivel básico [2], [3]. Una de sus características más destacadas es la presencia de 40 pines de propósito general (GPIO), los cuales permiten la interacción directa con componentes electrónicos externos como sensores, motores y pantallas [3].

El sistema operativo recomendado para la Raspberry Pi 4B es Raspberry Pi OS de 64 bits, el cual está basado en la distribución Linux Debian [1], [4]. Esta versión de 64 bits aprovecha directamente la arquitectura ARMv8-A del procesador BCM2711, permitiendo procesar datos en bloques de 64 bits de manera simultánea, lo que se traduce en un mejor rendimiento general del sistema frente a su equivalente de 32 bits [2], [5]. Una de sus ventajas más significativas es la capacidad de asignar toda la memoria RAM disponible a un único proceso, lo cual resulta especialmente relevante en aplicaciones de servidor o procesamiento intensivo [5]. El sistema operativo se instala en la tarjeta microSD de al menos 8 GB mediante la herramienta oficial Raspberry Pi Imager, desde la cual se selecciona la versión deseada que

posteriormente se graba en la tarjeta antes de insertarla en la placa [3], [5]. Una vez iniciado, Raspberry Pi OS ofrece un entorno de escritorio completo con navegador web, gestor de archivos, terminal y soporte para lenguajes de programación como Python [1], [4].

Para esta práctica se usó el lenguaje de programación Python con el fin de desarrollar los scripts de detección y validación, además se instalaron las librerías OpenCV, para el procesamiento de imágenes y captura de video; TensorFlow, para la ejecución del modelo de aprendizaje automático; NumPy, para el manejo de arreglos de datos en el procesamiento de imágenes. Se implementó además la herramienta de inteligencia artificial Google Teachable Machine, una plataforma no-code utilizada para recolectar el dataset y entrenar el modelo de clasificación utilizado en esta práctica.

II. METODOLOGÍA

En un inicio creamos una cuenta en el portal oficial de Raspberry Pi. El almacenamiento interno de la Raspberry Pi es la tarjeta microSD, la configuramos utilizando la herramienta Raspberry Pi Imager, primero la personalizamos asignándole el nombre ASE para identificar el dispositivo en la red y configuramos las credenciales de la red WIFI, generada desde el computador personal, a la cual se conectará automáticamente al encenderla, además habilitamos la opción *Raspberry Pi Connect* que permite el acceso remoto al dispositivo a través del servicio en la nube de Raspberry Pi, este servicio requiere la cuenta del portal oficial de Raspberry Pi, configuramos esta cuenta en la microSD usando una llave de autenticación, finalmente grabamos una imagen del sistema operativo Raspberry Pi OS de 64 directamente sobre la tarjeta desde el computador personal. Finalizado este proceso insertamos la microSD en la Raspberry Pi y verificamos su conexión a la red configurada.

La Raspberry Pi está vinculada a la cuenta desde el primer arranque, por ende, desde la cuenta podemos acceder a ella, identificando el dispositivo por su nombre y accediendo a él a través de la opción *Screen Sharing* que permite el control total del escritorio gráfico de la Raspberry Pi desde el navegador del computador personal.

Desde el acceso remoto preparamos el entorno de software necesario para el proyecto. Desde la terminal del sistema, se actualizaron los paquetes del sistema operativo a sus versiones más recientes con el fin de evitar posibles errores de compatibilidad entre dependencias. Luego, se creó un entorno virtual, *env_vision*, en Python 3.11.9, el entorno virtual garantiza que las librerías instaladas para este proyecto estén aisladas del resto del sistema operativo evitando conflictos

con otras instalaciones. En el entorno virtual se instalaron las librerías OpenCV, para el procesamiento de imágenes y captura de video; TensorFlow, para la ejecución del modelo de aprendizaje automático; NumPy, para el manejo de arreglos de datos en el procesamiento de imágenes. Para evitar errores forzamos la instalación de la versión `flatbuffers == 24.3.25`.

A continuación, se conectó una cámara USB estándar al dispositivo. Dado que el sistema operativo Linux incluye controladores UVC universales, no fue necesario instalar ningún driver adicional. Para comprobar que el sistema reconocía correctamente el dispositivo, se ejecutó el comando `ls /dev/video*`, el cual devolvió el nodo `/dev/video0`, confirmando la detección. Posteriormente, se instaló la herramienta `fswebcam` y se capturó una imagen de prueba para verificar el funcionamiento de la cámara. Adicionalmente, se ejecutó una prueba de humo que comprobó simultáneamente las versiones de OpenCV y TensorFlow instaladas, y verificó la correcta adquisición de un fotograma desde la cámara, generando un archivo de imagen como evidencia del proceso.

Para el entrenamiento del modelo de clasificación de imágenes utilizamos Google Teachable Machine, una plataforma no-code que permite construir modelos de redes neuronales de forma visual sin necesidad de código de entrenamiento. En esta plataforma se definieron tres clases de clasificación:

- **Fondo:** fondo vacío sin objetos con 37 fotos.
- **Cargador:** cabeza de un cargador USB con 77 fotos.
- **Memoria USB:** memoria USB estándar con 84 fotos.

Las imágenes se tomaron con una cámara priorizando los múltiples ángulos y posteriormente se subieron a la plataforma desde el computador personal. Una vez subidos los archivos se entrenó la red neuronal, dentro de la misma plataforma se verificó su funcionalidad y posteriormente se exportó el modelo en formato `keras.h5` compatible con TensorFlow y Python.

El modelo exportado se cargó en la Raspberry Pi. El script de inferencia fue desarrollado en Python y ejecutado desde el editor Thonny, configurado previamente para utilizar el intérprete del entorno virtual `env_vision`. El script realiza en tiempo real las siguientes operaciones: captura continua de fotogramas desde la cámara USB, redimensionamiento y normalización de cada imagen a las dimensiones esperadas por el modelo (224×224 píxeles), inferencia mediante TensorFlow y filtrado de predicciones con un umbral de confianza del 70 %, imprimiendo en consola el nombre de la clase detectada y el porcentaje de certeza correspondiente.

III. GRÁFICAS Y RESULTADOS

El modelo se entrenó con el fin de reconocer entre tres clases: el fondo, un cargador y una memoria USB. A continuación, se presentan los resultados obtenidos. La Tabla I resume las métricas promedio registradas durante las pruebas de cada clase.

En la Fig. 1 se pueden distinguir claramente los tres bloques de prueba por color. Los puntos azules (Cargador) son los más densos y generalmente altos ($> 90\%$), los naranjas (Fondo)

Cuadro I
RESUMEN DE LAS MÉTRICAS PROMEDIO POR CLASE.

Clase	Confianza	Latencia	CPU	RAM
Cargador	91.1 %	268.9 ms	53.2 %	46.2 %
USB	66.4 %	523.6 ms	49.0 %	46.1 %
Fondo	83.7 %	263.3 ms	53.4 %	46.0 %

también son altos, pero con mayor dispersión y los verdes (USB) presentan la mayor variación hacia abajo. Esto confirma visualmente que el Cargador es la clase más confiable y la USB la más problemática. Por otro lado, la latencia total evidencia que el pico inicial de ~ 3000 ms en el frame 0 corresponde al arranque del modelo. Se observa un segundo pico de ~ 600 ms alrededor del frame 40, que probablemente corresponde al primer frame de la segunda clase, asociado a un nuevo ciclo de inferencia o a la recolección de basura del sistema. El resto de la prueba se mantiene estable alrededor de 286 ms.

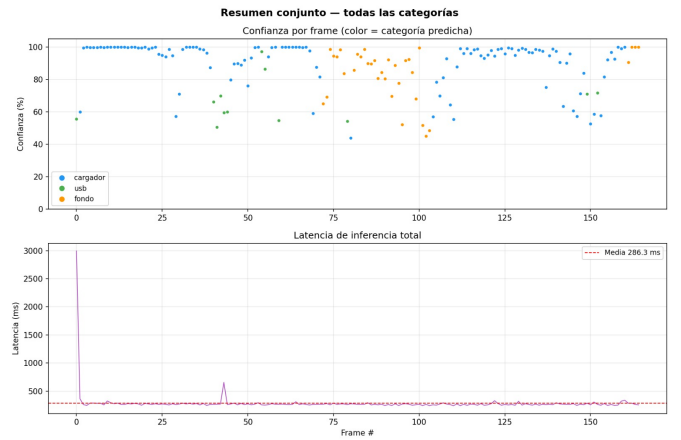


Figura 1. Gráficas análisis de las clases (Resumen conjunto).

III-A. Memoria USB

En la Fig. 2 se evidencia que la confianza oscila entre 50 % y 98 %, lo que indica que el modelo tiene dificultad para reconocer la USB de forma consistente, probablemente debido a que las imágenes de entrenamiento presentaban variaciones de ángulo y desenfoque.

Respecto a la latencia, esta arranca en aproximadamente 3000 ms por la inicialización del modelo y cae abruptamente a partir del frame 2. El promedio resultó más alto que el de las otras clases (523.6 ms), posiblemente porque las inferencias menos seguras requieren mayor tiempo de cómputo. Se observa además una caída puntual del CPU en el frame 4, la cual coincide con el valle de latencia en ese mismo punto, lo que sugiere que ese frame fue procesado con menor carga computacional.

El uso de CPU y RAM arranca bajo y sube rápidamente al estabilizarse, manteniéndose en 49.0 % y 46.1 % respectivamente durante el resto de la prueba.

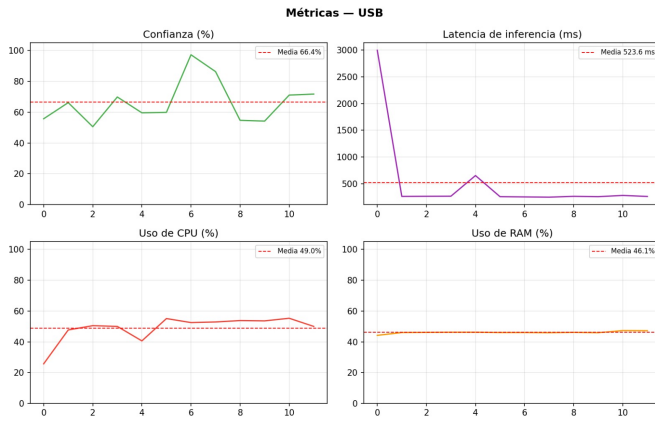


Figura 2. Gráficas análisis clase USB.

III-B. Cargador

El Cargador es la clase con mayor confianza promedio (91.1 %) y la que mejor clasifica el modelo. En la Fig. 3 se observa que, aunque hay valles pronunciados entre los frames 60 y 80 donde la confianza desciende hasta aproximadamente 44 %, el modelo se recupera rápidamente. Estos valles coinciden probablemente con ángulos o distancias inusuales del cargador frente a la cámara durante la prueba.

La latencia se mantiene estable y consistente durante la mayor parte de la prueba con un promedio de 268.9 ms, aunque aumenta ligeramente hacia los frames finales, lo que puede indicar acumulación de carga térmica en el procesador de la Raspberry Pi.

El uso de CPU es el más estable de las tres clases, con un promedio de 53.2 % y un pico final de aproximadamente 68 %, posiblemente atribuible a *throttling* térmico al final de una prueba prolongada. La RAM se mantiene prácticamente constante en 46.2 % durante todo el experimento, lo cual confirma que no hay fugas de memoria en el código.

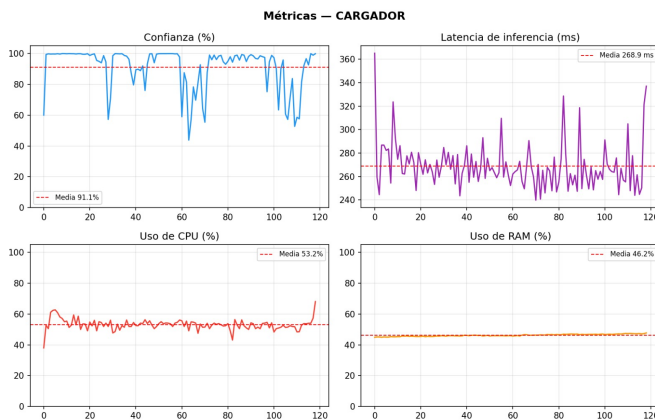


Figura 3. Gráficas análisis clase cargador.

III-C. Fondo

En la Fig. 4 se evidencian caídas notables en la confianza alrededor de los frames 20 y 25, lo que sugiere que en algunas capturas el fondo presentaba condiciones de iluminación o encuadre diferentes a las del conjunto de entrenamiento.

La latencia es la más baja y estable de las tres clases, oscilando entre 244 ms y 293 ms con un promedio de 263.3 ms. Este comportamiento es coherente con la naturaleza de la clase, ya que el fondo es visualmente la más simple, lo que permite al modelo resolver la predicción con mayor rapidez.

El uso de CPU y RAM se mantiene muy estable durante toda la prueba, con promedios de 53.4 % y 46.0 % respectivamente. Se registra un único pico de CPU del 65 % alrededor del frame 30. La RAM prácticamente no varía, lo que confirma que el modelo se encuentra completamente cargado en memoria desde el inicio de la prueba.

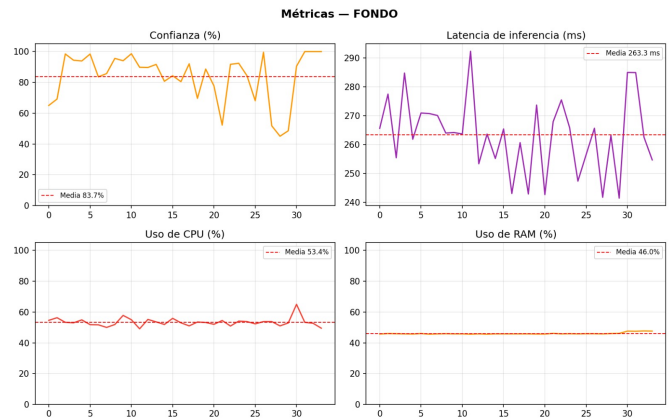


Figura 4. Gráficas análisis clase fondo.

IV. CONCLUSIONES

En general, el modelo presentó su mejor desempeño en la clase Cargador con un 91.1 % de confianza promedio, seguido de Fondo con 83.7 %. La clase USB registró el mayor margen de mejora con un 66.4 %, lo cual se atribuye a menor calidad y variedad de imágenes en su conjunto de entrenamiento, así como a las condiciones de desenfoque y cambio de ángulo presentes en las fotografías utilizadas. El uso de recursos del sistema se mantuvo estable en las tres pruebas, con un consumo de RAM constante cercano al 46 % y un CPU promedio entre 49 % y 53 %, lo que indica que el modelo opera dentro de los límites computacionales de la Raspberry Pi 4B sin comprometer su estabilidad.

REFERENCIAS

- [1] Raspberry Pi Foundation, "About us," Raspberry Pi Foundation, 2008. [En línea]. Disponible en: <https://www.raspberrypi.org/about/>. [Accedido: 20-may-2026].
- [2] Raspberry Pi Trading Ltd., *Raspberry Pi 4 Model B: Product Brief*, Cambridge, Reino Unido: Raspberry Pi Trading Ltd., 2020. [En línea]. Disponible en: <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-product-brief.pdf>. [Accedido: 20-may-2026].

- [3] Raspberry Pi Ltd., “Raspberry Pi 4 Model B – Specifications,” Raspberry Pi, 2019. [En línea]. Disponible en: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>. [Accedido: 20-may-2026].
- [4] Raspberry Pi Ltd., “Raspberry Pi OS (64-bit),” *Raspberry Pi Documentation*, 2022. [En línea]. Disponible en: <https://www.raspberrypi.com/documentation/computers/os.html>. [Accedido: 20-may-2026].
- [5] Raspberry Pi Ltd., “Raspberry Pi OS 64-bit,” *Raspberry Pi News*, feb. 2022. [En línea]. Disponible en: <https://www.raspberrypi.com/news/raspberry-pi-os-64-bit/>. [Accedido: 20-may-2026].